HATHOR-CORE · MASTER-BOOK

The Wire Format — Serialization

How a vertex becomes a compact, deterministic stream of bytes for the network and disk — Hathor's bespoke binary encoding, and why not JSON or Protobuf.

SUBJECT	hathor-core · Part II · the node, end to end
CHAPTER	26 · Part II · The Node
BRANCH	study/hathor-core-studies
EDITION	2026 · v1

Serialization · Binary encoding · Bytes · Endianness · Varint/LEB128 · Length-prefixing · Determinism · Funds vs graph · VertexParser

Table of Contents

Chapter 26 — The Wire Format: Serialization

What you'll learn in this chapter

- What **serialization** and **deserialization** are, framed as a general problem: how an object that lives in memory becomes a flat stream of bytes for the network or disk, and how it is reconstructed.
- The byte-level vocabulary a junior reader needs first — **bytes**, **endianness**, **fixed-width integers**, **variable-length integers (varint / LEB128)**, and **length-prefixing** — each defined and motivated before any Hathor code.
- The two serialization worlds inside `hathor-core`: the hand-rolled `struct`-based codec that turns a **vertex** into bytes, and the newer general-purpose `hathor/serialization/` framework (`Serializer` / `Deserializer` plus per-type encoders) that recent subsystems build on.
- Exactly how a vertex is laid out on the wire: the **funds struct**, the **graph struct**, the **nonce**, and the precise byte layouts of `TxInput` and `TxOutput`.
- How `VertexParser` reads raw bytes back into the correct vertex subclass using the version byte.
- **Why Hathor uses a bespoke binary format and not JSON or Protobuf** — the determinism that consensus and hashing demand, compactness, byte-exact cross-node reproducibility — and the honest cost of that choice.

Chapter 25 built the vertex classes — `Block`, `Transaction`, their inputs, outputs, and metadata — as objects living in the node's memory. It ended on a promise: those objects must eventually leave memory. A transaction you create has to travel across the network to your peers; a block the node accepts has to be written to disk so it survives a restart. Both directions require turning a Python object into a sequence of bytes, and turning those bytes back into the same object on the other side. That translation is **serialization**, and it is this chapter's whole subject.

Serialization is not a footnote in a blockchain. It is load-bearing in a way it never is in an ordinary web application, and the reason is the hash. A vertex's identity is the hash of its serialized bytes (Chapter 25, §25.5.2; the proof-of-work of Chapter 9 is computed over those same bytes). If two nodes serialized the same transaction into even slightly different byte strings, they would compute different hashes, disagree about the

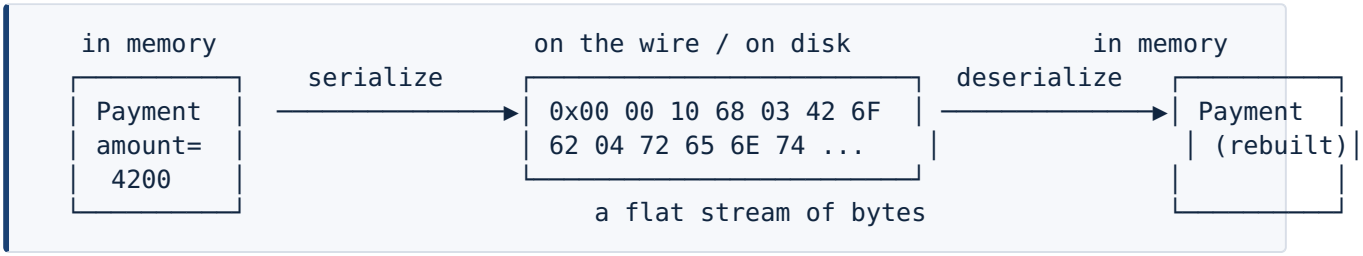
transaction's identity, and fall out of consensus. So the format cannot merely be a way to write the object down — it must be the one, exact, reproducible way, identical on every node, forever. That requirement shapes every decision in this chapter.

26.1 The problem, in general terms

Start away from Hathor entirely. You have an object in memory — say, in a hypothetical accounting program, a `Payment` with an amount, a recipient name, and a note:

```
payment = Payment(amount=4200, recipient="Bob", note="rent")
```

Inside the running program this is a tidy bundle of fields the language manages for you. But "in memory" is a fiction the moment you want to do anything outside this one process. A network socket does not carry Python objects; it carries bytes. A file on disk does not store fields; it stores bytes. To send the payment to another machine, or to save it, you must answer a flat question: **which exact sequence of bytes represents this object?** Producing that sequence is **serialization** (also called marshalling or encoding). The reverse — taking a byte sequence and rebuilding the object — is **deserialization** (unmarshalling, decoding, parsing).



The contract between the two sides is the **format**: a precise agreement about what each byte means and in what order the bytes appear. Get the format right on both ends and the rebuilt object equals the original. Get it wrong by a single byte and you get garbage, a crash, or — worse for a blockchain — a different but plausible-looking object.

Any serialization format has to make four decisions, and the rest of this chapter is really just Hathor's answers to them:

1. **How are numbers written as bytes?** A Python `int` has no fixed size; bytes do. You must choose how many bytes a number occupies, and in which order those bytes go.
2. **How are variable-length things framed?** A script or a note can be any length. The reader, scanning left to right, must know where one field ends and the next begins.
3. **How are the fields ordered, and which is which?** The reader rebuilds fields in a fixed sequence; the writer must emit them in that same sequence.

4. **How does the reader know which kind of object it is reading?** A stream of bytes is just bytes; something in it must say "this is a `Block`" versus "this is a `Transaction`."

Before Hathor's answers, the vocabulary those answers use.

26.2 The concepts it rests on

Bytes, and why everything reduces to them

A **byte**[1] is eight bits — a number from 0 to 255. Every file, every network packet, every disk block is, at bottom, a sequence of bytes. Higher-level things (text, numbers, objects) are interpretations layered on top of bytes by agreement. Python gives you a `bytes` object (immutable) and a `bytearray` (mutable) to hold such sequences; serialization is the craft of building one of these from an object, and reading one back.

Endianness — the order of a number's bytes

A single byte holds 0–255, which is rarely enough; a timestamp or a coin value needs several bytes. The moment a number spans more than one byte, a question appears that has no "natural" answer: **in what order do the bytes go?** Consider the number 4200, which in hexadecimal is `0x1068`. It needs two bytes: `0x10` and `0x68`. There are two ways to lay them down:

```
the number 4200 = 0x1068, written in two bytes:

big-endian     :  0x10 0x68      ← most-significant byte first ("big end" first)
little-endian  :  0x68 0x10      ← least-significant byte first ("little end" first)
```

This choice is called **endianness**[2]. Neither order is more correct; what matters is that the writer and reader agree. Big-endian is often called "network byte order" because most internet protocols use it, and it reads naturally left-to-right like we write decimal numbers. **Hathor's vertex format is big-endian throughout** — its low-level `int_to_bytes` helper hard-codes `byteorder='big'` (`hathor/transaction/util.py:32`), and the `struct` format strings start with `!` (network/big-endian order, e.g. `_GRAPH_FORMAT_STRING = '!dIB'`, `base_transaction.py:72`). The general serialization framework's integer encoder also hard-codes big-endian (`encoding/int.py:48`). Whenever you see a multi-byte number in this chapter, read it big end first.

Fixed-width integers — spending exactly N bytes

The simplest way to write a number is to decide up front how many bytes it gets and always use that many. A timestamp gets 4 bytes; a length counter gets 1 byte; a hash gets 32 bytes. This is a **fixed-width** encoding. Its virtue is that the reader always

knows exactly how far to advance. Its cost is waste: the number 5 stored in 4 bytes still occupies 4 bytes (`00 00 00 05`), three of them zero. The vertex format uses fixed-width integers for fields whose range it knows in advance — which is most of them.

Variable-length integers (varint / LEB128) — paying for what you use

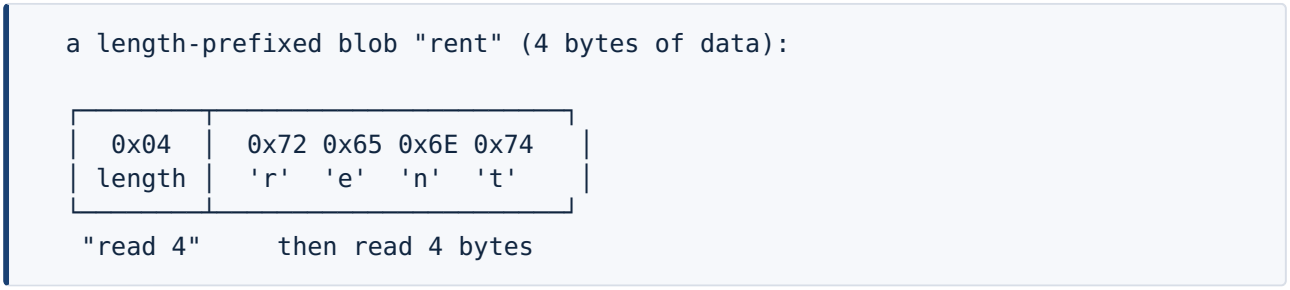
When a number's range is wide but its typical value is small, fixed width wastes space. The fix is a **variable-length integer**, or **varint**[3]: an encoding that uses one byte for small numbers and grows only as needed. The standard scheme — the one Hathor's general framework uses — is **LEB128**[4] ("Little Endian Base 128"). The idea is straightforward once you see it:

- Chop the number into 7-bit groups.
- Store each group in the low 7 bits of a byte.
- Use the high bit of each byte as a **continuation flag**: `1` means "another byte follows," `0` means "this is the last byte."

So a number under 128 fits in a single byte (high bit `0`); a number up to 16,383 takes two bytes; and so on. The reader keeps consuming bytes until it meets one whose high bit is `0`. Hathor's implementation is `encode_leb128` / `decode_leb128` (`hathor/serialization/encoding/leb128.py:54` and `:76`); the continuation bit is the `| 0b1000_0000` on every non-final byte (`:73`) and the `& 0b1000_0000` test that stops the reader (`:90`). (The "signed" variant additionally checks the sign bit `0b0100_0000`, so it can encode negative numbers too.)

Length-prefixing — framing variable-length data

The last primitive idea is **length-prefixing**. When a field can be any length — a script, a note, a list of inputs — the reader scanning left-to-right needs to know where it ends before it reads it. The universal trick: write the length first, then the data. The reader reads the length, then reads exactly that many bytes.



The alternative — a delimiter, like the null byte that ends a C string — fails for binary data, because the data itself might contain the delimiter byte. Length-prefixing has no such ambush: the length is separate metadata, and the data that follows is opaque.

Hathor length-prefixes every variable field. The only thing that varies is how the length itself is written — a fixed-width count in the vertex codec, a LEB128 count in the general framework — as we will see.

> **Recap — the vertex (full treatment in Ch. 25).** A vertex is any node of Hathor's ledger DAG — a `Block` or a `Transaction`. Each carries a `version`, `signal_bits`, `weight`, `timestamp`, a `nonce`, a list of `inputs` (`TxInput`, the UTXO spending pointers), a list of `outputs` (`TxOutput`, the coins it creates), and a list of `parents` (the DAG topology edge). A transaction additionally carries a `tokens` list. Those are the fields this chapter turns into bytes. → full treatment in Ch. 25.

> **Recap — the hash and determinism (full treatment in Ch. 9 & 25).** A vertex's identity is the double-SHA256 of its serialized bytes; the proof-of-work nonce is the number a miner varies so that this hash falls under a target. Because the hash is computed over the serialized bytes, the serialization must be byte-for-byte identical on every node. Serialization is therefore not a convenience layer — it is part of the consensus rules. → full treatment in Ch. 9 (proof-of-work) and Ch. 25 (the hash invariant).

26.3 Localization

Two parts of the tree do serialization. The first is the general-purpose framework, the package this chapter is named for:

```
hathor/
└── serialization/                ◀ YOU ARE HERE — the general encoding framework
    │   serializer.py             ← Serializer (ABC): write_byte / write_bytes / write_struct
    │   deserializer.py           ← Deserializer (ABC): read_byte / read_bytes / peek_* / rea
    │   bytes_serializer.py       ← BytesSerializer (concrete: accumulate into memory)
    │   bytes_deserializer.py     ← BytesDeserializer (concrete: read from a memoryview)
    │   types.py                  ← Buffer = bytes | memoryview
    │   consts.py                 ← DEFAULT_LEB128_MAX_BYTES, DEFAULT_BYTES_MAX_LENGTH
    │   exceptions.py             ← SerializationError + OutOfDataError / BadDataError / TooL
    │
    ├── encoding/                 ← per-primitive encoders (each a pair of free functions)
    │   │   int.py                ← fixed-width signed/unsigned ints (big-endian)
    │   │   leb128.py             ← variable-length ints (the canonical varint)
    │   │   bytes.py              ← length-prefixed byte blobs (length is LEB128)
    │   │   bool.py               ← a bool as one byte (0x00 / 0x01)
    │   │   utf8.py               ← length-prefixed UTF-8 text
    │   │   output_value.py       ← the clever coin-value codec (4-or-8 bytes)
    │   │   ecv.py                ← elliptic-curve value encoding
    │
    ├── compound_encoding/        ← encoders for shapes built from primitives
    │   │   optional.py           ← "maybe a value" (a presence flag + the value)
    │   │   collection.py         ← length-prefixed lists/sets
    │   │   mapping.py            ← length-prefixed dicts
    │   │   tuple.py              ← fixed-arity tuples
    │   │   signed_data.py        ← a payload plus its signature
    │
    └── adapters/                 ← wrappers that constrain an encoder
        │   max_bytes.py          ← caps how many bytes a field may consume
        │   generic_adapter.py    ← base for such wrapping adapters
```

The second is older, and lives inside the transaction package — the hand-rolled codec the vertex classes use:

```
hathor/
└── transaction/
    │   base_transaction.py       ← get_funds_struct / get_graph_struct / get_struct,
    │                                __bytes__, TxInput.__bytes__, TxOutput.__bytes__
    │   transaction.py            ← Transaction.get_funds_struct + parse-back methods
    │   block.py                  ← Block.get_funds_struct + parse-back methods
    │   vertex_parser.py          ← VertexParser.deserialize: bytes → right subclass
    │   util.py                   ← int_to_bytes, unpack, output_value_to_bytes
```

> **Context.** Two serialization worlds coexist, and that is the central fact to hold while reading this chapter. The **vertex wire format** — the bytes that get hashed, sent to peers, and stored — is produced by hand-written code in `hathor/transaction/` using Python's built-in `struct` module and a few helpers in `transaction/util.py`. The newer **`hathor/serialization/` framework** is a general, reusable encoding toolkit (a `Serializer` / `Deserializer` pair plus per-type encoders) that more recent subsystems — nano-contracts, vertex headers — build on, and into which the vertex codec's value encoding has already been migrated (`transaction/util.py` calls `decode_output_value` from the new package). Read §26.4 for the framework's design and §26.5 for the vertex format itself; they meet at the coin-value codec.

26.4 The general framework, walked

Build the intuition with a hand-encoded toy first, then meet the real classes.

26.4.1 A toy encoding, by hand

Suppose you must serialize a tiny record: one integer `id` and one variable-length blob `name`. You decide the format yourself:

- `id` → a 2-byte big-endian unsigned integer.
- `name` → a length-prefixed blob: one byte of length, then that many bytes.

Encoding `id=4200, name=b"Bob"` by hand:

```
id = 4200 = 0x1068, big-endian, 2 bytes      →  0x10 0x68
len(name) = 3, one byte                       →  0x03
name = b"Bob"                                 →  0x42 0x6F 0x62

full stream:  10 68 03 42 6F 62        (6 bytes)
```

Decoding reverses it with the format as the only guide: read 2 bytes as a big-endian int → 4200; read 1 byte → 3; read 3 bytes → `b"Bob"`. Notice that the decoder is a cursor moving left to right, consuming exactly as many bytes as the format dictates at each step. Every serializer in the world, Hathor's included, is this cursor idea dressed up. The `hathor/serialization/` framework's only ambition is to give you reusable, named pieces for "read 2 bytes as an int" and "read a length-prefixed blob" so you do not hand-roll the cursor every time.

26.4.2 `Serializer` and `Deserializer` — the two cursors

The framework is built around a matched pair, both declared as **abstract base classes**[5] (so the byte-level mechanics can have more than one concrete implementation). The **`Serializer`** (`hathor/serialization/serializer.py:32`) is the writing cursor: its concrete subclass `BytesSerializer` accumulates the output as a list

of byte parts (`bytes_serializer.py:21`). The **`Deserializer`** (`hathor/serialization/deserializer.py:32`) is the reading cursor: its concrete subclass `BytesDeserializer` holds the input as a `memoryview` and shortens it as bytes are consumed (`bytes_deserializer.py:24`). They are deliberate mirror images — whatever you write, you can read back.

The two abstract methods every serializer must provide are tiny:

```python
@abstractmethod
def write_byte(self, data: int) -> None:
    """Write a single byte."""

@abstractmethod
def write_bytes(self, data: Buffer) -> None:
    ...
```

(`serializer.py:41`, `:46`.) The concrete `BytesSerializer` implements them by appending to its internal list of `memoryview` parts and joining them only at the end, in `finalize()` (`bytes_serializer.py:33`). This "defer the join until the end" detail is a performance choice — concatenating bytes repeatedly is costly, so it keeps the pieces apart and joins once.

The deserializer's core methods come in two flavours, **peek** and **read**. A read consumes bytes and advances the cursor; a peek looks without consuming:

```python
@abstractmethod
def peek_byte(self) -> int:
    """Read a single byte but don't consume from buffer."""

@abstractmethod
def read_byte(self) -> int:
    """Read a single byte as unsigned int."""

@abstractmethod
def read_bytes(self, n: int, *, exact: bool = True) -> Buffer:
    """Read n bytes, when exact=True it errors if there isn't enough data"""
```

(`deserializer.py:47`, `:62`, `:67`.) The `peek` capability matters more than it looks: the coin-value codec in §26.5.3 peeks at the first byte to decide whether a value is 4 or 8 bytes long before it commits to reading. The `exact=True` default means "if there are fewer than `n` bytes left, raise" — a malformed stream is rejected, not silently truncated. The concrete reader raises `OutOfDataError` (`bytes_deserializer.py:47`) in that case.

There are also two `struct`-aware convenience methods that bridge to Python's built-in `struct` module: `write_struct(data, format)` packs a tuple per a format string, and `read_struct(format)` / `peek_struct(format)` unpack one (`serializer.py:52`, `deserializer.py:87`, `:56`). These are the seam where the new framework reuses the same `struct` machinery the old vertex codec uses directly.

26.4.3 The primitive encoders

The `encoding/` modules are the framework's real content. Each is a pair of free functions, `encode_*` and `decode_*`, that take the serializer (or deserializer) as their first argument and call its `write_*` / `read_*` methods. This is a deliberate design: the cursor classes know only how to move bytes; the meaning of each type lives in a small, independently-testable function. Each module's docstring even contains runnable doctest examples — a good place to see the exact bytes each encoder produces.

Fixed-width integers (`encoding/int.py`). A thin, safe wrapper over Python's built-in `int.to_bytes`:

```
def encode_int(serializer: Serializer, number: int, *, length: int, signed: bool) -> None:
    try:
        data = int.to_bytes(number, length, byteorder='big', signed=signed)
    except OverflowError:
        raise ValueError('too big to encode')
    serializer.write_bytes(data)
```

(`int.py:42`.) Two things to note. First, the byte order is hard-coded to `'big'` — there is one endianness for the whole format, so it can never accidentally mix orders. Second, the `OverflowError` that `to_bytes` raises when a value does not fit in `length` bytes is translated into a `ValueError` (`:50`), so the caller sees a clean "won't fit" failure rather than a leaked low-level exception. The decoder mirrors it with `int.from_bytes(..., byteorder='big', signed=signed)` (`int.py:60`).

Variable-length integers (`encoding/leb128.py`). This is the canonical varint from §26.2. The encoder loops, peeling off 7 bits at a time and setting the continuation bit on every byte but the last:

```
def encode_leb128(serializer: Serializer, value: int, *, signed: bool) -> None:
    if not signed and value < 0:
        raise ValueError('cannot encode value <0 as unsigend')
    while True:
        byte = value & 0b0111_1111          # low 7 bits of the remaining value
        value >>= 7                          # shift them off
        if signed:
            cont = (value == 0 and (byte & 0b0100_0000) == 0) or \
                   (value == -1 and (byte & 0b0100_0000) != 0)
        else:
            cont = (value == 0 and (byte & 0b1000_0000) == 0)
        if cont:
            serializer.write_byte(byte)          # last byte: high bit stays 0
            break
        serializer.write_byte(byte | 0b1000_0000)  # more to come: set continuation bit
```

(`leb128.py:54`.) The decoder mirrors it, reassembling the value 7 bits at a time and stopping when it meets a byte with the high bit clear — the `if (byte & 0b1000_0000) == 0` test at `leb128.py:90`, after which it reads any remaining bytes via `read_byte` in a

loop. (Note: the current `encode_leb128` takes no length cap; the repository keeps the recommended cap as a constant, `DEFAULT_LEB128_MAX_BYTES = 4` in `consts.py:15`, applied by callers that want it — see §26.6.)

Length-prefixed byte blobs (`encoding/bytes.py`). This is §26.2's length-prefixing made literal, and it composes the two integer encoders cleanly:

```
def encode_bytes(serializer: Serializer, data: bytes) -> None:
    encode_leb128(serializer, len(data), signed=False)
    serializer.write_bytes(data)

def decode_bytes(deserializer: Deserializer) -> bytes:
    size = decode_leb128(deserializer, signed=False)
    return bytes(deserializer.read_bytes(size))
```

(`bytes.py:67`, `:77`.) The length goes first as an unsigned LEB128, then the raw bytes. The reader does the reverse: read the length, then read exactly that many bytes. (The module docstring works a nice example: a 4-byte blob gets a 1-byte length prefix `\x04`, while a 128-byte blob gets a 2-byte length prefix `\x80\x01` — the varint growing only when it must.)

Booleans (`encoding/bool.py`) are one byte: `0x01` for true, `0x00` for false, and any other byte is rejected as invalid (`encode_bool` / `decode_bool`, `bool.py:61`, `:68`). **Text** (`encoding/utf8.py`) is encoded to UTF-8 and then length-prefixed exactly like `bytes`.

26.4.4 The compound encoders

The `compound_encoding/` package builds shapes out of those primitives, and these are where serialization stops being about single values and starts being about whole data structures.

- **Optional** (`compound_encoding/optional.py`) handles "a value that might be absent." The trick from §26.1: write a one-byte presence flag, and only if it is set, write the value — composed from whatever encoder handles the inner value.
- **Collection** (`compound_encoding/collection.py`) handles lists and sets: length-prefix the count, then encode each item.
- **Mapping** (`compound_encoding/mapping.py`) does the same for dictionaries (count, then key/value pairs); **tuple** does it for fixed-arity tuples; **signed_data** pairs a payload with its signature.

The pattern throughout is composition: a compound encoder knows the shape (a count followed by items) and is given a function for the element. This is how the framework stays general without a giant table of every concrete type — the higher-order-function style of Chapter 2 applied to bytes.

26.4.5 The adapters

The `adapters/` package holds wrappers that constrain an existing cursor rather than define a new one. The clearest is `max_bytes` (`adapters/max_bytes.py`): `Serializer.with_max_bytes(n)` returns a wrapper that enforces no more than `n` bytes are written, and `Deserializer.with_max_bytes(n)` enforces no more than `n` are read (`serializer.py:56`, `deserializer.py:92`). Conceptually this is the **adapter / decorator** idea from Chapter 3 and Chapter 4 — interpose a wrapper that adds a check without rewriting the thing it wraps. Such caps exist because a deserializer reads attacker-supplied bytes; a field allowed to be "any length" is a denial-of-service waiting to happen (§26.6).

26.5 The vertex format, walked

Now the format that actually matters for consensus: how a `Block` or `Transaction` becomes the exact bytes that get hashed, sent, and stored. This path predates the framework of §26.4 and is hand-written in `hathor/transaction/` using Python's `struct` module and the helpers in `transaction/util.py`. It is the canonical example of a bespoke binary format, so we walk it in full.

26.5.1 The two structs: funds and graph

A serialized vertex is built in two halves, mirroring the two kinds of edge from Chapter 25. The split is not cosmetic; it reflects that a vertex has a value aspect (what coins it moves) and a topology aspect (where it sits in the DAG).

The **funds struct** carries the value side — the header bytes, the counts, and the inputs, outputs, and tokens themselves. For a `Transaction` (`transaction.py:205`):

```
def get_funds_struct(self) -> bytes:
    struct_bytes = pack(
        _FUNDS_FORMAT_STRING,      # '!BBBBB' — five unsigned bytes, big-endian
        self.signal_bits,
        self.version,
        len(self.tokens),
        len(self.inputs),
        len(self.outputs),
    )
    for token_uid in self.tokens:
        struct_bytes += token_uid        # 32 bytes each
    for tx_input in self.inputs:
        struct_bytes += bytes(tx_input)  # §26.5.3
    for tx_output in self.outputs:
        struct_bytes += bytes(tx_output) # §26.5.3
    return struct_bytes
```

Read the layout off the code directly. Five single-byte header fields come first, in this exact order: `signal_bits`, `version`, the **token** count, the **input** count, and the **output** count (`_FUNDS_FORMAT_STRING = '!BBBBB'`, `transaction.py:44`). Then the bodies, in the

same order the counts appeared: the token UIDs (32 bytes each), then the inputs, then the outputs. The matching reader, `get_funds_fields_from_struct` (`transaction.py:163`), unpacks the same five header bytes and then loops the recorded number of times to rebuild each list:

```
(self.signal_bits, self.version, tokens_len, inputs_len, outputs_len), buf = \
    unpack(_FUNDS_FORMAT_STRING, buf)
for _ in range(tokens_len):
    token_uid, buf = unpack_len(TX_HASH_SIZE, buf)   # 32 bytes
    self.tokens.append(token_uid)
for _ in range(inputs_len):
    txin, buf = TxInput.create_from_bytes(buf, verbose=verbose)
    self.inputs.append(txin)
for _ in range(outputs_len):
    txout, buf = TxOutput.create_from_bytes(buf, verbose=verbose)
    self.outputs.append(txout)
```

(`transaction.py:177`–`:201`.) The counts are why this works: because the writer prefixed each list with its length (the length-prefixing idea of §26.2, applied to lists of structures rather than blobs), the reader knows exactly how many times to loop. A `Block` overrides `get_funds_struct` to a shorter three-field header — `signal_bits`, `version`, output count — then just the outputs, with no inputs and no tokens (`block.py:222`; `block.py`'s `_FUNDS_FORMAT_STRING = '!BBB'` at `:39`) — because a block mints coins, never spends them, and only ever holds HTR (Chapter 25, §25.5.3). The version byte tells the reader which of these two layouts to expect.

> **A subtlety worth pausing on — field order versus the base class.** The base `GenericVertex.get_funds_struct` is abstract (`base_transaction.py:438`, just `raise NotImplementedError`): the base class deliberately refuses to define a funds layout, because the two concrete kinds differ. The on-wire truth is therefore always a concrete subclass's method — `Transaction` writes `signal_bits, version, #tokens, #inputs, #outputs`; `Block` writes `signal_bits, version, #outputs`. Both put `signal_bits` first and `version` second, which is exactly what the parser relies on: `VertexParser.deserialize` reads the version from `data[1]`, the second byte (§26.5.4). When in doubt, trust the subclass that runs and the parser that reads — they are the authority on what the bytes mean.

The **graph struct** carries the topology side — the proof-of-work weight, the timestamp, and the parents (`base_transaction.py:442`):

```
def get_graph_struct(self) -> bytes:
    struct_bytes = pack(_GRAPH_FORMAT_STRING, self.weight, self.timestamp, len(self.parents))
    for parent in self.parents:
        struct_bytes += parent           # 32 bytes each
    return struct_bytes
```

The format string is `_GRAPH_FORMAT_STRING = '!dIB'` (`base_transaction.py:72`): a `d` (8-byte double — the weight), an `I` (4-byte unsigned int — the timestamp), and a `B` (1-byte parent count), all big-endian. Then each parent hash follows verbatim, 32 bytes apiece — fixed, so no per-parent length prefix is needed (a hash is always exactly 32 bytes). The reader is `get_graph_fields_from_struct` (`base_transaction.py:413`), which unpacks `'!dIB'` and then reads `parents_len` hashes of `TX_HASH_SIZE` (32) bytes each (`:424`–`:432`). That the weight is serialized as a `double` is notable: weight is logically `log2(work)`, a real number by nature (Chapter 9), so a floating-point field is the honest representation.

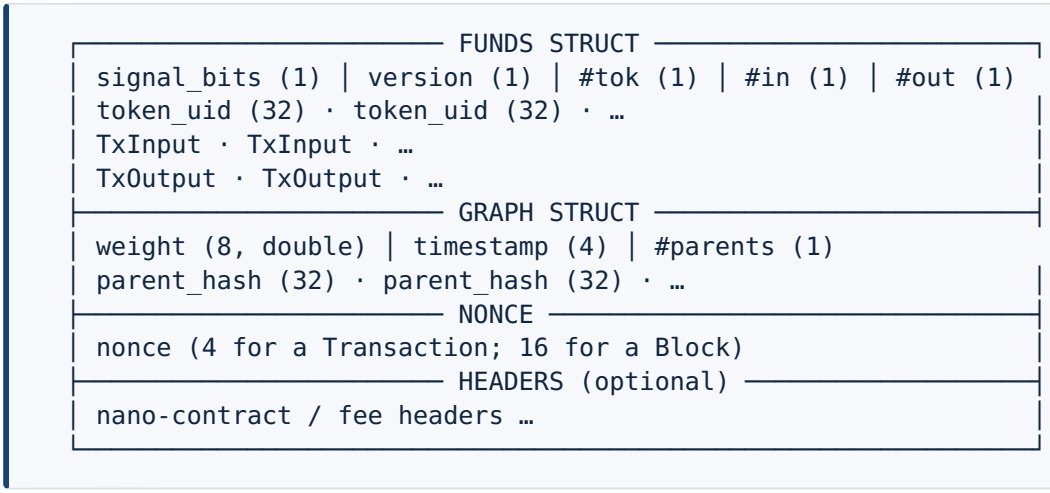
26.5.2 Assembling the whole vertex

The two structs plus the proof-of-work nonce plus any headers make the full serialization (`base_transaction.py:477`):

```python
def get_struct(self) -> bytes:
    struct_bytes  = self.get_struct_without_nonce()   # funds struct + graph struct
    struct_bytes += self.get_struct_nonce()           # the nonce
    struct_bytes += self.get_headers_struct()         # optional trailing headers
    return struct_bytes

def __bytes__(self) -> bytes:
    return self.get_struct()
```

`get_struct_without_nonce` (`:457`) concatenates the funds struct then the graph struct. `get_struct_nonce` (`:467`) writes the nonce in `SERIALIZATION_NONCE_SIZE` bytes — and that size **differs by vertex kind**: a `Transaction` uses **4 bytes** (`transaction.py:58`), a `Block` uses **16 bytes** (`block.py:46`), because a block's proof-of-work search space is much larger. `get_headers_struct` (`:453`) appends any optional trailing headers (nano-contract, fee). The whole thing is reachable through Python's `bytes(vertex)` because `__bytes__` is defined (`:329`) — so anywhere the codebase needs a vertex's bytes, it writes `bytes(tx)`, and that is the on-wire form. The full byte layout of a transaction:



This same byte string is what the node hashes for the vertex's identity: the hash is the double-SHA256 built from a mining header without the nonce (`get_mining_header_without_nonce`, `base_transaction.py:593`) folded together with the nonce (`calculate_hash` at `:628`, via `calculate_hash1` / `calculate_hash2` at `:603` / `:613`). Because the hash is taken over exactly these bytes, the layout is part of the protocol — change one field's width or order and every hash in the ledger would change.

26.5.3 `TxInput` and `TxOutput` byte layouts

The inputs and outputs each serialize themselves through their own `__bytes__`. A **`TxInput`** (`base_transaction.py:959`) is the UTXO spending pointer of Chapter 7:

```
def __bytes__(self) -> bytes:
    ret  = self.tx_id                       # 32 bytes — which prior tx
    ret += int_to_bytes(self.index, 1)      # 1 byte  — which output of it
    ret += int_to_bytes(len(self.data), 2)  # 2 bytes — unlocking-script length
    ret += self.data                        # the unlocking script itself
    return ret
```

Layout: a 32-byte transaction hash, a 1-byte output index, a 2-byte length, then the unlocking-script bytes. The `(tx_id, index)` pair is the pointer "spend output `index` of transaction `tx_id`"; `data` is the signature-and-public-key that satisfies that output's lock. Note the length-prefixing again — the script is variable, so its length comes first, here as a fixed 2-byte field (max 65,535 bytes). The reader `TxInput.create_from_bytes` reverses it: 32 bytes for the hash, then `unpack('!BH', buf)` for the 1-byte index and 2-byte length, then that many script bytes (`:983`–`:998`).

A nearby method earns a mention because Chapter 25 referenced it: `get_sighash_bytes` (`:971`) serializes an input with its `data` blanked to zero length (`int_to_bytes(0, 2)`, `:980`). This is the signing trick — you cannot sign over the very signature you are about to produce, so the body that gets signed uses empty unlocking scripts.

A **`TxOutput`** (`base_transaction.py:1022`) is a coin (`__bytes__` at `:1069`):

```
def __bytes__(self) -> bytes:
    ret  = output_value_to_bytes(self.value)   # 4 OR 8 bytes (see below)
    ret += int_to_bytes(self.token_data, 1)    # 1 byte — token index + authority bit
    ret += int_to_bytes(len(self.script), 2)   # 2 bytes — locking-script length
    ret += self.script                         # the locking script itself
    return ret
```

Layout: the value (variable, 4 or 8 bytes — next paragraph), the one `token_data` byte (Chapter 25, §25.5.3: low 7 bits pick the token, high bit flags an authority output — `TOKEN_INDEX_MASK` / `TOKEN_AUTHORITY_MASK` at `:1025` / `:1026`), a 2-byte script length, then the locking-script bytes. The reader is symmetric (`create_from_bytes`, `:1081`): decode the value, `unpack('!BH', buf)` for the token_data byte and the script length, then the script.

The value field is the most interesting micro-encoding in the whole vertex format, and it is the bridge into the §26.4 framework. A coin value is usually small enough for 4 bytes, but Hathor must allow values up to 2^{63}, which needs 8. Rather than always spending 8 bytes, the codec writes small values in 4 bytes and large values in 8, and steals the **sign bit** to tell them apart. `transaction/util.py`'s `bytes_to_output_value` now delegates to the framework's `decode_output_value`, whose encoder partner is `encode_output_value` (`hathor/serialization/encoding/output_value.py:89`):

```
MAX_OUTPUT_VALUE_32 = 2 ** 31 - 1     # fits in 4 signed bytes
MAX_OUTPUT_VALUE_64 = 2 ** 63         # the absolute maximum

def encode_output_value(serializer: Serializer, number: int, *, strict: bool = True) -> None:
    if number < 0:
        raise ValueError('Number must not be negative')
    if strict and number == 0:
        raise ValueError('Number must be strictly positive')
    if number > MAX_OUTPUT_VALUE_64:
        raise ValueError(...)
    if number > MAX_OUTPUT_VALUE_32:
        serializer.write_bytes((-number).to_bytes(8, byteorder='big', signed=True))  # 8-byte for
    else:
        serializer.write_bytes(number.to_bytes(4, byteorder='big', signed=True))     # 4-byte for
```

The mechanism: a positive value that fits in 4 bytes is written as a positive 4-byte signed integer — its top bit is `0`. A value too big for that is written as its negation in 8 bytes — a negative number, whose top bit is `1`. The decoder peeks at the first byte (`decode_output_value`, `:108`): if it is negative (`value_high_byte < 0`), the value was the 8-byte negated form, so read 8 bytes (`'!q'`) and negate back; otherwise read 4 bytes (`'!i'`) as-is. One bit, no separate length field, and the common case stays compact. The decoder even rejects a value that was written in 8 bytes but would have fit in 4 (`:125`) — enforcing the one canonical encoding rule that §26.6 is about. This is the kind of byte-thrift a bespoke format buys you and a generic one rarely does.

26.5.4 `VertexParser` — bytes back into the right subclass

Serialization's hard half is deserialization, because the reader is handed raw, possibly-hostile bytes and must decide what they are before it can parse them. Hathor answers the §26.1 question "which kind of object is this?" with the **version byte**. The entry point is `VertexParser.deserialize` (`hathor/transaction/vertex_parser.py:53`):

```python
def deserialize(self, data: bytes, storage: TransactionStorage | None = None) -> BaseTransaction:
    """ Creates the correct tx subclass from a sequence of bytes """
    from hathor.transaction import TxVersion
    version = data[1]                       # the version field is the SECOND byte
    try:
        tx_version = TxVersion(version)
        is_valid = self._settings.CONSENSUS_ALGORITHM.is_vertex_version_valid(
            tx_version, include_genesis=True, settings=self._settings,
        )
        if not is_valid:
            raise StructError(f"invalid vertex version: {tx_version}")
        cls = tx_version.get_cls()          # dispatch: version → concrete class
        return cls.create_from_struct(data, storage=storage)
    except ValueError as e:
        raise StructError('Invalid bytes to create transaction subclass.') from e
```

The flow is: read the version from `data[1]` (the second byte — recall the wire order is `signal_bits` then `version`, §26.5.1), confirm the running network's consensus algorithm actually permits that version (a PoA network, for instance, rejects regular blocks), look up the matching class through `TxVersion.get_cls()` (`base_transaction.py:115` — the dispatch table mapping `REGULAR_BLOCK → Block`, `REGULAR_TRANSACTION → Transaction`, `TOKEN_CREATION_TRANSACTION → TokenCreationTransaction`, `MERGE_MINED_BLOCK → MergeMinedBlock`, `POA_BLOCK → PoaBlock`, and — only when nano-contracts are enabled — `ON_CHAIN_BLUEPRINT → OnChainBlueprint`), and hand the whole byte string to that class's `create_from_struct`. That method (`transaction.py:143`) constructs an empty instance, calls the `get_*_fields_from_struct` readers in funds-then-graph order, reads the nonce (`unpack('!I', buf)` — 4 bytes for a transaction, `:152`), then loops `get_header_from_bytes` for any trailing headers, and finally `update_hash()`. An unknown or disallowed version raises `StructError`, so a malformed or future-versioned vertex is rejected cleanly rather than misinterpreted. This single dispatch[6] point is why every vertex must carry its own version: it is the one piece of self-description in an otherwise positional, schema-less format.

26.5.5 The two worlds, reconciled

You have now seen both serialization paths and the seam between them. The vertex codec (§26.5) is hand-written, positional, and `struct`-based; it is frozen because the bytes it produces are hashed and therefore part of consensus — you cannot change it without changing every vertex's identity. The general framework (§26.4) is newer, composable, and the home of reusable encoders; newer subsystems (nano-contract data, vertex headers) build on it, and the vertex codec's value encoding has already moved into it (`output_value.py`, called back from `transaction/util.py`). The honest summary: Hathor is mid-migration. The framework is where new serialization should live; the vertex format is a stable island that the framework is slowly absorbing one codec at a time. For a reader, the rule is simple — for the bytes that get hashed, read `transaction/`; for everything newer, read `serialization/`.

26.6 Why bespoke binary, and not JSON or Protobuf

This is the question the whole chapter exists to answer, and it is a real engineering decision with real trade-offs, not a foregone conclusion. A node could, in principle, represent a transaction as JSON (`{"version": 1, "inputs": [...], ...}`) or use a schema tool like Protobuf[7]. Hathor uses neither. Here is the case, and the cost.

1. Determinism — the decisive reason. A vertex's identity is the hash of its bytes (§26.2, §26.5.2), and consensus requires every node to compute the same hash for the same vertex. That demands a **canonical** encoding: exactly one byte string per object, reproducible on every machine, in every language, forever. JSON fails this outright. JSON does not fix the order of object keys, the whitespace, the number formatting (`4200` vs `4.2e3`), or the handling of large integers — two correct JSON encoders can emit different bytes for the same data, and then their hashes differ and the nodes disagree. Protobuf is closer but still does not guarantee a canonical byte form across implementations (field ordering and default-value handling can vary; its spec explicitly does not promise deterministic output). A hand-specified positional binary format has no such freedom: there is one order, one width, one endianness for every field — and the codec even rejects a value encoded in 8 bytes that would have fit in 4 (`output_value.py:125`), closing the last door to two encodings of one number. The bytes are the spec. This is why blockchains across the board use bespoke binary formats for the data that gets hashed. **Determinism is not a nice-to-have here; it is the requirement that rules out the alternatives.**

2. Compactness. A node stores and ships millions of vertices. The binary form is dense: a 4-byte timestamp instead of a 20-character ISO string, a 1-byte count instead of `"inputs":[`, a small coin value in 4 bytes instead of a decimal string. JSON would inflate every field with field names, quotes, brackets, and — because binary like a 32-byte hash is not representable in JSON at all without an encoding such as hex or base64 — would roughly double the size of every hash. Over the whole ledger and over every byte sent to every peer, that overhead is large. Compactness directly lowers disk usage, bandwidth, and sync time.

3. Byte-exact reproducibility and bounded parsing. Because the format is positional and self-contained, parsing is a straight left-to-right cursor walk with no reflection and no schema registry — and re-serializing a parsed vertex yields the identical bytes, which is exactly what the hash check relies on. The deserializer also stays defensive cheaply: `read_bytes(..., exact=True)` raises `OutOfDataError` rather than returning short data (`bytes_deserializer.py:65`), the `with_max_bytes` adapters cap how much a field may consume (`deserializer.py:92`), and the recommended LEB128 length cap (`consts.py:15`) keeps a length field from claiming an absurd size. A hostile peer cannot send a tiny message that claims a multi-gigabyte field and exhaust the node's memory. A general JSON/Protobuf parser is harder to bound this tightly.

Now the honest cost, because every choice has one:

- **You maintain the format by hand.** There is no `.proto` schema generating reader and writer for you; `get_funds_struct` and `get_funds_fields_from_struct` are two hand-written functions that must stay in lockstep. If they drift, vertices serialize one way and parse another, and the bug is silent until a hash mismatches.
- **No human-readability.** You cannot open a stored vertex in a text editor and read it; you need the format in your head or a decoder in your hand. (This is partly why `TxInput` / `TxOutput` keep `to_human_readable` methods that render to a JSON-friendly dict — for APIs and debugging, not for the wire.)
- **Versioning is manual.** Adding a field means bumping the version byte and teaching the parser the new layout — there is no schema-evolution machinery doing it for you. Hathor's `version` field and the optional trailing `headers` (§26.5.2) are the deliberate seams that make this manageable, but it is still hand-work.

The trade is the classic systems trade: a bespoke binary format costs developer effort and readability to buy determinism, density, and bounded speed. For data whose hash is its identity and whose encoding is part of the consensus rules, that trade is not close — the things you give up are conveniences, and the thing you gain is correctness.

26.7 How it plugs into the lifecycle

Serialization sits at every boundary where a vertex crosses out of, or into, the node's memory.

- **Creating and signing.** When the wallet builds a transaction, it serializes the body with each input's `data` blanked (`get_sighash_bytes`, §26.5.3) to produce the sighash the owner signs (Chapter 25) — serialization is already in play before the vertex is even complete.
- **Hashing and mining.** The proof-of-work nonce is searched against the hash of the serialized bytes; mining a block is, mechanically, re-serializing with different nonces until the hash meets the target (Chapter 9, Chapter 37).
- **Sending to peers.** When the node ships a vertex over a P2P connection, it sends `bytes(vertex)` and the peer reconstructs it with `VertexParser.deserialize` (Chapter 34). The two nodes agree on the object precisely because they agree on the bytes.
- **Writing to and reading from storage.** When a vertex is persisted, it is stored as these bytes and read back through the same parser. That is the subject of the next chapter — the format you just learned is exactly what lands in RocksDB.

- **APIs.** Read-only surfaces (the JSON/HTTP and WebSocket feeds) expose a vertex both as a hex string of its serialized bytes and, via `to_human_readable`, as a friendly dict — decoded through the same paths.

At every one of these boundaries, the invariant from the chapter's opening holds: the bytes are canonical, so the hash is canonical, so every node agrees.

Recap

Decision (§26.1)	Hathor's answer	Where
How numbers become bytes	big-endian throughout; fixed-width for known ranges, LEB128 varint in the framework	`transaction/util.py:32`; `base_transaction.py:72`; `encoding/int.py:42`; `encoding/leb128.py:54`
Framing variable-length data	length-prefix everything (2-byte fixed length for scripts; 1-byte counts for lists; LEB128 in the framework)	`TxInput.__bytes__` `base_transaction.py:959`; `TxOutput.__bytes__` `:1069`; `bytes.py:67`
Vertex field order	funds struct (signal_bits, version, counts, tokens/in/out) then graph struct (weight, timestamp, parents) then nonce then headers	`transaction.py:205`; `base_transaction.py:442`, `:477`
Which kind of object	the version byte at `data[1]`, dispatched through `TxVersion.get_cls()`	`vertex_parser.py:53`; `base_transaction.py:115`
The two cursors (framework)	`Serializer` (write_) / `Deserializer` (read_/peek_*), with free `encode_*` / `decode_*` functions	`serializer.py:32`; `deserializer.py:32`
Coin value codec	4 bytes if $\leq 2^{31}-1$, else 8 bytes negated; the sign bit is the flag	`serialization/encoding/output_value.py:89`
Nonce size	4 bytes for a `Transaction`, 16 bytes for a `Block`	`transaction.py:58`; `block.py:46`
Why bespoke binary	determinism for hashing/consensus, compactness, bounded parsing — at the cost of hand-maintenance and no readability	§26.6

Serialization is where Chapter 25's objects stop being Python and become the protocol. A vertex's bytes are produced by a hand-written, positional, big-endian format in `hathor/transaction/` — a funds struct, a graph struct, a nonce, and optional headers — with `TxInput` and `TxOutput` laying out their own bytes and a one-bit trick keeping coin values compact. `VertexParser` reads those bytes back into the right subclass by dispatching on the version byte. Alongside it, the newer `hathor/serialization/` framework offers a reusable `Serializer` / `Deserializer` toolkit, built from small `encode_*` / `decode_*` functions, that the rest of the node is migrating onto. The format is bespoke and binary, not JSON or Protobuf, for one reason above all the others: the hash is taken over these exact bytes, so the encoding must be canonical and reproducible on every node — determinism is the whole game. With the bytes of a vertex now in hand, the next chapter follows them to their resting place: Chapter 27, persistence, where these byte strings are written to and read from RocksDB.

---

1. A byte is eight bits, holding an integer from 0 to 255. All files and network data are sequences of bytes; everything else — text, numbers, objects — is an interpretation layered on top of bytes by agreement. ↩
2. Endianness is the order in which the bytes of a multi-byte number are laid down. Big-endian puts the most-significant byte first (the way we write decimal); little-endian puts the least-significant byte first. Neither is more correct; writer and reader need only agree. Hathor uses big-endian everywhere. ↩
3. A varint (variable-length integer) is an integer encoding that uses fewer bytes for smaller numbers — one byte for small values, more only as needed — instead of a fixed width. It saves space when most values are small. ↩
4. LEB128 ("Little Endian Base 128") is the standard varint scheme Hathor's framework uses: the number is split into 7-bit groups, each stored in the low 7 bits of a byte, with the high bit of each byte acting as a "more bytes follow" flag. The reader stops at the first byte whose high bit is 0. It is the same scheme used by WebAssembly and the DWARF debug format. ↩
5. An abstract base class (ABC) is a class you cannot instantiate directly; it declares methods (often `@abstractmethod`) that concrete subclasses must implement. `Serializer` and `Deserializer` are ABCs, with `BytesSerializer` / `BytesDeserializer` as the concrete in-memory implementations. Full treatment in Chapter 1. ↩
6. Dispatch means selecting which piece of code to run based on a value at runtime — here, choosing a vertex class based on the version byte. Full treatment of dispatch as a pattern is in Chapter 4. ↩
7. Protobuf (Protocol Buffers) is Google's schema-driven binary serialization tool: you write a `.proto` schema and it generates reader/writer code. It is compact and convenient, but its output is not guaranteed to be byte-for-byte canonical across implementations, which is why it is unsuitable for data whose hash must match on every node. ↩